

Squirtle Squad

CENTSIBLE

TEST PLAN

Sprint 4

Date: 07/12/2026

Table of Contents

1	<u>Introduction</u>	1
	<u>Objectives</u>	1
	<u>Team Members</u>	1
2	<u>Scope</u>	1
	<i><u>Sprint 2</u></i>	1
	<i><u>Sprint 3</u></i>	1
	<i><u>Sprint 4 (current)</u></i>	1
3	<u>Assumptions / Risks</u>	1
	<u>Assumptions</u>	1
	<u>Risks</u>	1
4	<u>Test Approach</u>	1
	<i><u>Sprint 3</u></i>	1
	<i><u>Sprint 4 (current)</u></i>	1
	<i><u>Sprint 5</u></i>	1
	<u>Test Automation</u>	1
	<u>Test Cases (Black Box)</u>	1
	<i><u>Feature: Authentication & Access Control</u></i>	1
	<i><u>Feature: Transaction Management</u></i>	1
	<i><u>Feature: Budget Management</u></i>	1
	<i><u>Feature: Bills & Subscriptions</u></i>	1
	<i><u>Feature: Dashboard & Reporting</u></i>	1
	<u>Test Cases (White Box)</u>	1
	<u>Priority</u>	1
	<u>Test Cases</u>	1
5	<u>Test Environment</u>	1
6	<u>Milestones / Deliverables</u>	1
	<u>Test Schedule</u>	1
	<u>Deliverables</u>	1
	<u>Defects / Bugs Identified This Sprint</u>	1

1 Introduction

The Test Plan has been created to communicate the test approach to team members. It includes the objectives, scope, schedule, risks, and approach. This document identifies the test deliverables and what is in and out of scope.

Objectives

Centsible is a personal finance tracker that lets a user manage income and expenses, set monthly budgets, track recurring bills and subscriptions, and view a summary dashboard and reports.

Sprint 4 delivered backend CRUD support for bills and subscriptions (FR4), closing a gap that existed since Sprint 2: the database schema for these features existed, but no API routes did. This sprint also fixed a data-integrity bug in the default category seeding. Testing this sprint covers every feature built to date across all sprints, not just what changed.

Team Members

Resource Name	Role (examples are given below)
Hoang Bui	Developer/ Tester
Andre Phanavanh (Team Lead)	Project Manager/ Developer
Austin Ridge	Developer/ Tester
Muhammad Taha	Developer/ Tester

Remaining roles to be filled in by the rest of the team. Roles: Developer, Project Manager, Tester.

2 Scope

All "Must Have" and "Should Have" requirements from the SRS are included below. The scope of testing is determined at the beginning of each sprint; this document keeps a record of each sprint's scope as the project progresses.

Sprint 2

At the end of Sprint 2, a user must be able to:

- Create, view, edit, and delete income and expense transactions (FR2, US3).
- Set a monthly budget limit per category (FR3, US2).

Recorded retroactively for reference; detailed test cases from Sprint 2 are not repeated here.

Sprint 3

At the end of Sprint 3, a user must be able to:

- View the dashboard summarizing balance, income, expenses, and recent transactions (FR5, US3).
- View a monthly breakdown of spending by category on the Reports page (US5).

Recorded retroactively for reference; detailed test cases from Sprint 3 are not repeated here.

Sprint 4 (current)

At the end of Sprint 4, a user must be able to:

- Register and log in with a name, email, and password (FR1, US3). Verified working; session expiration and route-level access control are not yet implemented, see Risk #1 and #2.
- Create, view, edit, and delete income and expense transactions with amount, date, category, type, and description (FR2, US3). Verified working.
- Set a monthly budget limit per category and see current spending calculated against it (FR3, US2). Verified working.
- Add bills with a name, amount, and due date, and track subscriptions on a billing cycle (FR4, US1, US4). Backend verified working this sprint; frontend still displays mock data, see Risk #3.
- View a dashboard summarizing balance, income, expenses, recent transactions, and budget progress (FR5, US3). Verified working, except the Bills Due Soon widget.
- View a monthly breakdown of spending by category on the Reports page (US5). Verified working.

Security and validation testing was also conducted: input validation, duplicate-data handling, cross-user data scoping, and role-based access control (guest vs. standard user).

3 Assumptions / Risks

Assumptions

- The team continues using SQLite locally for all environments through the end of the class project.
- Grading and demos happen against the local dev setup (npm run dev on both backend and frontend), not a deployed production build.
- The JWT and bcrypt auth stack specified in Design Doc 2 is still the intended direction for real authentication, it just hasn't been implemented yet.

Risks

#	Risk	Impact	Trigger	Mitigation Plan
1	No token/session-based authentication exists. Every route trusts a raw userId passed by the client, so any request can read or modify another user's data by changing that value.	High	Before final delivery / any real deployment	Design Doc 2 already specifies JWT + bcrypt as the intended auth stack. Implement JWT issuance on login/register and require a valid token on every protected route before Sprint 5.
2	No route guard on the frontend. A user can type /dashboard directly into the browser without logging in.	Medium	Demo or grading walkthrough	Add a simple auth check in App.jsx that redirects to /login when no user is stored, ahead of the full JWT work if needed as a stopgap.
3	Bills and Subscriptions pages, plus the Dashboard's "Bills Due Soon" widget, still render hardcoded mock arrays instead of calling the new /api/bills and /api/subscriptions endpoints.	Medium	Any grading pass that clicks into the Bills page	Backend CRUD is complete and tested. Wire Bills.jsx and Dashboard.jsx to the real endpoints next.
4	No automated test suite exists. All testing this sprint was manual/API-level.	Medium	Any future regression in a route or the schema	Add a lightweight test runner (Node's built-in test runner or Jest) for backend routes, starting with the ones covered manually in this document.

5	No rate limiting on /api/auth/login. Nothing prevents repeated password-guessing attempts.	Low	Would only matter in a real deployment, not local grading	Add a basic rate limiter (e.g. express-rate-limit) on the auth routes before any deployment outside class.
6	Weekly delivery may slip because team members work on different schedules.	Medium	Missed sprint deadline	Keep scope for each sprint small and testable in isolation, as done this sprint with bills/subscriptions being additive only.

4 Test Approach

The project is using an agile approach, with 2-week sprints. Testing during each sprint is conducted manually and, where applicable, at the API level.

Sprint 3

Manual walkthrough of the dashboard and reports pages against seeded data, confirming totals matched the underlying transactions.

Sprint 4 (current)

Testing was conducted manually and at the API level (via curl against the running backend), plus a full manual walkthrough of the built frontend pages. Every result in Section 4.2 was actually executed against a running instance of the app, not simulated.

Sprint 5

Planned: automated test coverage for backend routes (see Risk #4), plus manual verification once Bills.jsx and Dashboard.jsx are connected to real data (see Risk #3).

Test Automation

No automated test suite exists yet for either the backend or frontend. All results in this document come from manual and API-level testing. Adding automated coverage is planned; see Risk #4.

Test Cases (Black Box)

User-facing test cases: functional requirements, UI behavior, and role-based access control, exercised the way an end user would. Grouped by feature, matching each User Story area in the SRS.

Feature: Authentication & Access Control

Test Case ID	Description	Requirements Trace	Directions	Expected Output	Actual Result	Status	Issues / Bugs
TC-101	Register a new user with valid name, email, and password.	FR1	1. Go to /register. 2. Enter name, email, password (8+ chars). 3. Submit.	Account is created, user is logged in and redirected to the dashboard.	Matched expected exactly.	Pass	None.
TC-102	Register with a password under 8 characters.	FR1	1. Go to /register. 2. Enter a 6-character password. 3. Submit.	Form is rejected with a password-length error.	Matched expected exactly (blocked client-side and server-side).	Pass	None.

TC-103	Register with an email that already has an account.	FR1	1. Register once. 2. Register again with the same email.	Second registration is rejected, no duplicate account created.	409 error returned: "That record already exists."	Pass	None.
TC-104	Log in with correct email and password.	FR1, US3	1. Go to /login. 2. Enter valid credentials. 3. Submit.	User is logged in and redirected to the dashboard.	Matched expected exactly.	Pass	None.
TC-105	Log in with a wrong password.	FR1	1. Go to /login. 2. Enter valid email, wrong password. 3. Submit.	Generic error shown, no indication whether the email exists.	Matched expected exactly.	Pass	None.
TC-106	Load the dashboard directly in the browser without logging in first (role-based access control).	FR1, Role 2 (Guest)	1. Open a fresh browser session. 2. Navigate directly to /dashboard.	Guest is redirected to /login; dashboard content is not accessible.	No redirect exists. The dashboard layout renders for a guest and only fails quietly when its data calls run with no user id.	Fail	Bug: missing frontend route guard. Logged as Risk #2.
TC-503	Attempt to submit the registration form with an empty required field.	FR1	1. Leave the password field blank. 2. Submit.	Form is blocked, no request sent.	Matched expected exactly.	Pass	None.

Feature: Transaction Management

Test Case ID	Description	Requirements Trace	Directions	Expected Output	Actual Result	Status	Issues / Bugs
TC-201	Add an expense transaction through the Transactions page.	FR2, US3	1. Go to /transactions. 2. Fill in amount, category, date, description. 3. Submit.	Transaction appears at the top of the list immediately.	Matched expected exactly.	Pass	None.
TC-204	Edit an existing transaction.	FR2	1. Click edit on a transaction. 2. Change the amount. 3. Save.	Transaction updates in place in the list.	Matched expected exactly.	Pass	None.
TC-205	Delete a transaction.	FR2	1. Click delete on a transaction. 2. Confirm.	Transaction is removed from the list.	Matched expected exactly.	Pass	None.

Feature: Budget Management

Test Case ID	Description	Requirements Trace	Directions	Expected Output	Actual Result	Status	Issues / Bugs
TC-301	Set a monthly budget limit for a category.	FR3, US2	1. Go to /budget. 2. Choose a category and enter a monthly limit. 3. Save.	Budget appears with a progress bar reflecting current spending.	Matched expected exactly.	Pass	None.

Feature: Bills & Subscriptions

Test Case ID	Description	Requirements Trace	Directions	Expected Output	Actual Result	Status	Issues / Bugs
TC-401	Add a bill with a name, amount, and due date.	FR4, US4	1. Go to /bills. 2. Fill in the Add Bill form. 3. Submit.	New bill appears in the Upcoming Bills list.	Form accepts the input, but the list does not update because it is not	Fail	Same root cause as TC-407. Logged as Risk #3.

					connected to real data.		
TC-407	Confirm the Bills & Subscriptions page reflects real account data.	FR4, US1	1. Create a bill via the API. 2. Reload /bills.	The bill created via the API appears in the Upcoming Bills list.	Page shows a fixed hardcoded list regardless of what exists in the account.	Fail	Frontend not yet wired to /api/bills or /api/subscriptions. Logged as Risk #3.
TC-408	Confirm the Dashboard's "Bills Due Soon" widget reflects real account data.	FR5, US1	Same as TC-407, then check the dashboard.	Widget reflects the user's actual upcoming bills.	Widget shows a hardcoded mock array, same as TC-407.	Fail	Same root cause as TC-407. Logged as Risk #3.

Feature: Dashboard & Reporting

Test Case ID	Description	Requirements Trace	Directions	Expected Output	Actual Result	Status	Issues / Bugs
TC-501	Load the dashboard and confirm balance, income, expense, and budget-left cards populate.	FR5, US3	1. Log in with an account that has transactions and a budget. 2. View /dashboard.	All four summary cards show live values.	Matched expected exactly.	Pass	None.
TC-502	Load the Reports page and confirm category breakdown and top transactions populate.	FR5, US5	1. View /reports on the same account.	Category breakdown and top 5 transactions render from live data.	Matched expected exactly.	Pass	None.

Test Cases (White Box)

Implementation-aware test cases: API-level input validation, data-integrity, and cross-user scoping checks.

Test Case ID	Description	Test Steps	Expected Output	Actual Result	Status	Issues / Bugs
TC-107	Confirm sessions expire after inactivity, per FR1 / NFR1.	1. Log in. 2. Leave the session idle. 3. Check whether the login state ever expires.	Session or token expires and the user is prompted to log in again.	No session or token layer exists at all; login state never expires client-side.	Fail	Gap against FR1 and NFR1. Logged as Risk #1.
TC-202	POST /api/transactions with an invalid type value.	{"type":"transfer","amount":10,...}	400 error, transaction not created.	400 returned: "type must be 'income' or 'expense'."	Pass	None.
TC-203	POST /api/transactions with a negative amount.	{"amount":-10,...}	400 error, transaction not created.	400 returned: "amount must be a number >= 0."	Pass	None.
TC-206	GET /api/transactions/:id using a different user's id.	Valid transaction id, unrelated userId in the query string.	404, not found, no data leak.	Matched expected exactly (though scoping relies on a trusted client-supplied id, see Risk #1).	Pass	None.
TC-302	POST /api/budgets twice for the same user/category/month/year with different limits.	Same period, monthly_limit changed from 300 to 250.	Existing row is updated (upsert), not duplicated.	Matched expected exactly, same budget id returned both times.	Pass	None.

TC-303	POST /api/budgets with an out-of-range month.	{"month":13,...}	400 error.	400 returned: "month must be an integer 1-12."	Pass	None.
TC-304	GET /api/budgets and confirm current_spending is computed correctly.	One \$300 budget, one \$45.50 expense in the same category/month.	current_spending returns 45.50, calculated on read from transactions.	Matched expected exactly.	Pass	None.
TC-402	POST /api/bills with an invalid frequency value.	{"frequency":"biweekly",...}	400 error, bill not created.	400 returned listing the allowed frequency values.	Pass	None.
TC-404	POST /api/subscriptions without an is_active field.	No is_active field sent.	is_active defaults to 1 (true).	Matched expected exactly.	Pass	None.
TC-405	DELETE /api/bills/:id, then GET the same id.	DELETE then GET on the same bill id.	204 No Content, then 404 on the follow-up GET.	Matched expected exactly.	Pass	None.
TC-406	GET /api/subscriptions/:id using a different user's id.	Valid subscription id, unrelated userId.	404, not found, no data leak.	Matched expected exactly.	Pass	None.
TC-601	Insert a duplicate global (default) category directly at the database layer.	Second INSERT of category name 'Income' with user_id NULL.	Insert is rejected, only one 'Income' default category ever exists.	Before fix: silently succeeded. After adding a partial unique index on categories(name) WHERE user_id IS NULL: correctly throws a UNIQUE constraint error.	Pass (after fix)	Bug found and fixed this sprint, see schema.sql.
TC-602	Submit a non-numeric value in an amount field across transactions, budgets, and bills routes.	amount: "fifty"	400 error, request rejected before hitting the database.	Matched expected exactly on all three routes.	Pass	None.
TC-603	Full backend install and startup on a clean machine.	npm install, db:migrate, db:seed, npm run dev	Server starts and listens on port 3000 with no errors.	Matched expected exactly.	Pass	None.
TC-604	Full frontend production build.	npm run build	Vite build completes with no errors.	Matched expected exactly, build completed in ~1.2s.	Pass	None.

Priority

Test cases are prioritized to guide what gets re-run first if time is short in a future sprint:

- High: covers a "Must Have" requirement, or any currently-failing test case (session expiration, route guard, bill/subscription frontend wiring).
- Medium: covers a "Should Have" requirement that is currently passing.
- Low: covers build/install/environment checks (TC-603, TC-604) and "Could Have" scope.

Test Cases

Test cases are organized by feature area above, matching the directory structure planned for an automated test suite: backend/tests/auth, backend/tests/transactions, backend/tests/budgets, backend/tests/bills, backend/tests/subscriptions, one file per feature, named to match the Test Case IDs in this document (e.g. TC-4xx tests live in backend/tests/bills).

5 Test Environment

Backend: Node.js, Express 5, better-sqlite3, running locally on port 3000. Frontend: React 19 with Vite, running locally on Vite's dev server port, communicating with the backend over `http://localhost:3000/api`. Database: a local SQLite file, reset and reseeded between test passes with `npm run db:reset`.

6 Milestones / Deliverables

Test Schedule

Task Name	Start	Finish	Effort	Comments
Build bills & subscriptions backend routes	7/9/2026	7/11/2026	~4 hrs	Complete, tested this sprint.
Manual/API-level testing of Sprint 4 features	7/11/2026	7/12/2026	~3 hrs	Complete, results in Section 4.
Fix duplicate default-category schema bug	7/12/2026	7/12/2026	~30 min	Complete (TC-601).
Wire Bills.jsx and Dashboard.jsx to real endpoints	7/13/2026	7/17/2026	~3 hrs	Not started. See Risk #3.
Add frontend auth route guard	7/13/2026	7/14/2026	~2 hrs	Not started. See Risk #2.
Implement JWT-based authentication	7/15/2026	7/22/2026	~6 hrs	Not started. See Risk #1.
Add automated test coverage for backend routes	7/20/2026	7/26/2026	~5 hrs	Not started. See Risk #4.

Deliverables

- This Test Plan document (living, updated each sprint).
- Working backend routes for bills and subscriptions, merged via pull request.
- Updated schema.sql with the default-category uniqueness fix.

Defects / Bugs Identified This Sprint

- Fixed: duplicate default categories could be inserted because SQLite treats every NULL as a distinct value, so the original `UNIQUE(user_id, name)` constraint never caught two global (`user_id IS NULL`) rows with the same name. Resolved with a partial unique index in schema.sql (TC-601).
- Open: no frontend route guard, a guest can view the dashboard layout without logging in (TC-106).
- Open: no session/token expiration exists despite being required by FR1 and NFR1 (TC-107).
- Open: Bills.jsx, the Dashboard's Bills Due Soon widget, and services/api.js all still reference mock data instead of the new `/api/bills` and `/api/subscriptions` endpoints (TC-401, TC-407, TC-408).

This is a living document and will be updated as testing continues in future sprints.